

3 - TIER ARCHITECTURE

Prepared by:

R.VINAY KUMAR

Mail Id:

vinaykumarrepalle93@gmail.com

Batch: 149-Offline



Fig 1: 3-TIER ARCHITECTURE

Overview

AWS provides a wide range of resources for developing and managing cloud applications, which can be customized to construct highly dependable and resilient cloud infrastructures. Suppose you are tasked with developing a three-tier architecture that is readily available for your organization's new web application.

The 3-tier architecture is a software architecture pattern where the application is broken down into three logical tiers: presentation layer, business logic layer and the data storage layer. It is used in a client-server application such as a web application with the frontend, backend and the database.

1. Web Tier (Presentation Layer)

- It handles user interface and client requests
- The user is going to access only frontend part
- The presentation tier receives the response from the application tier

- The presentation tier receives the response from the application tier

2. Application Tier (Business Logic Layer)

- It handles the application's core processing, business logic
- It is the brain of the application
- It processes user requests, applies rules, and makes decisions.

3. Data Tier (Storage Layer)

- It reliably stores the application's data and handles all access requests
- It protects data integrity and ensures consistency
- Storage and manage the data in database tier

Benefits of a 3 Tire Architecture:

Scalability: is the ability of a system, application, or network to handle increasing workload or users without losing performance.

Reliability: is the ability of a system or application to work correctly and consistently over time without failures.

Performance: is how fast and efficiently a system or application responds to user requests and completes tasks.

Security: is the practice of protecting a system, data, and resources from unauthorized access, misuse, or attacks.

Reduced development time: building a software application faster by saving time during design, coding, testing, and deployment.

Flexibility: is the ability of a system to adapt easily to changes without major effort or cost.

- **Step 1: Network Setup**

- **Create a Virtual Private Cloud (VPC)**
Define CIDR block and enable DNS resolution & DNS hostname.
Create two public subnets and four private subnets across multiple Availability Zones.
- **Attach Internet Gateway**
Connect to Public subnets for external access.
- **Configure Route Tables**
Public subnets route through Internet Gateway.
Private subnets route through NAT Gateway.

Before creating a VPC, log in to the AWS Management Console using the Root user email and password, then use the saved AWS passkey to continue.
Select one Region like (Oregon) (us-west-2).

Process

1. Create the VPC:

- Search VPC it navigates to the Amazon VPC dashboard
- Select Your VPCs -> Create VPC
- Select VPC and more for a comprehensive setup
- Provide a name tag for auto-generation->**Project-vpc**
- Specify an IPv4 CIDR block->20.0.0.0/16

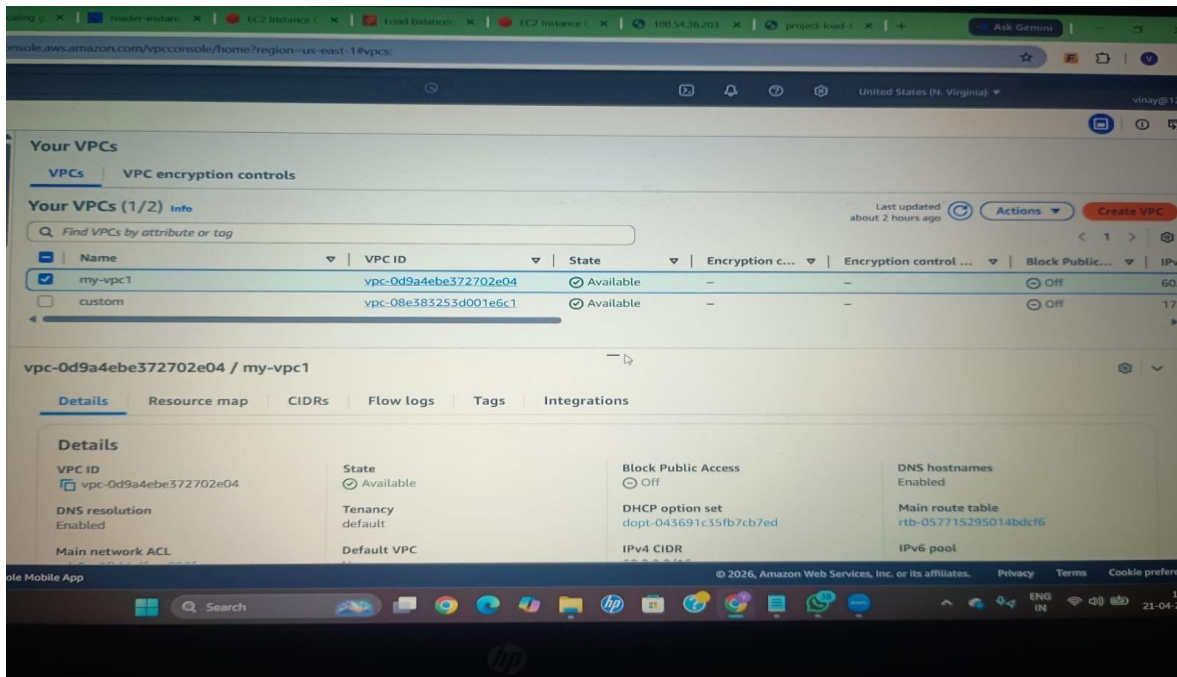


Fig 1.1: VPC

2. Configure Subnets:

- Determine the number of Availability zones to use (2).
- Specify the number of public subnets (2) and private subnets (4) needed.
- Review and customize the CIDR blocks for each subnet if required.

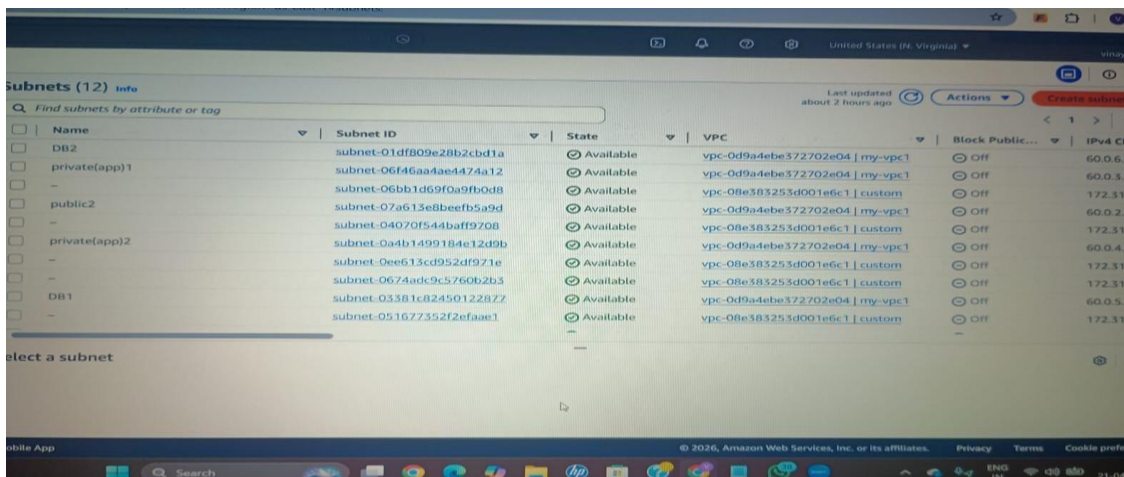


Fig 1.2: Subnets

3. Configure Internet Gateway:

- For public subnets to access the internet an Internet Gateway is required.

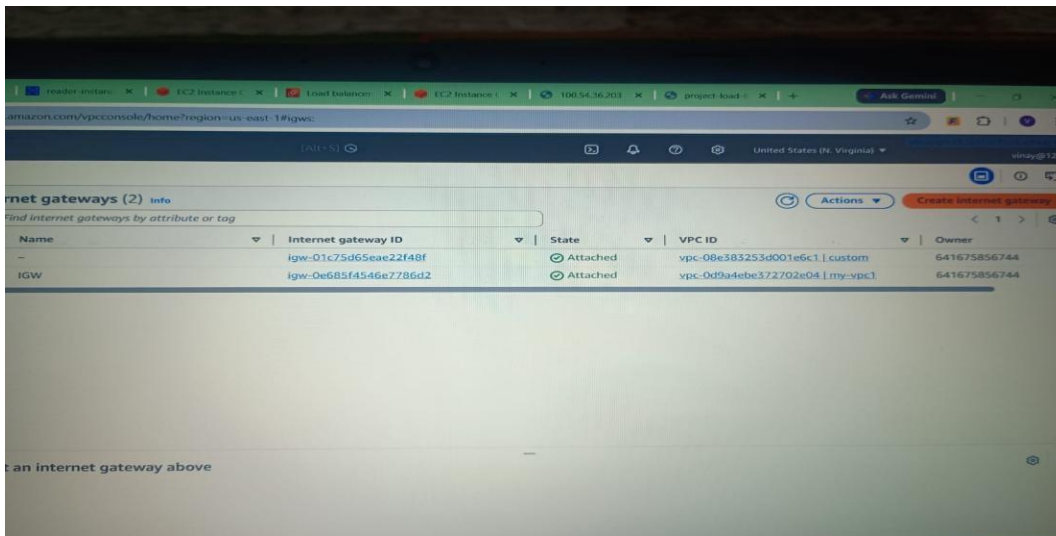
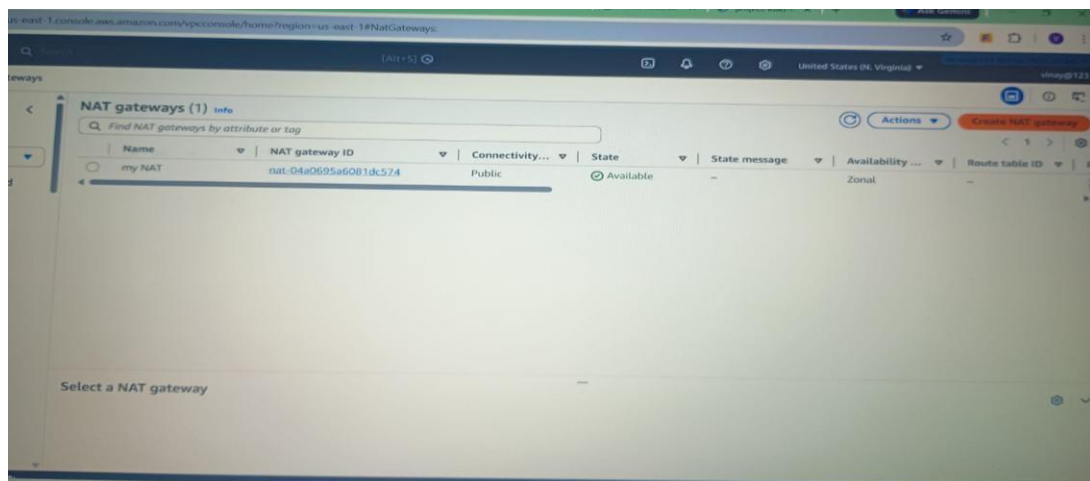


Fig 1.3: Internet Gateway

4. Configure NAT Gateway

- Create 2 NAT Gateway: Project-NAT-Gateway-1, Project-NAT-Gateway-2
- For private subnets need outbound internet access is necessary
- Create Allocate elastic Ip for both NAT Gateway

Fig 1.4: NAT Gateway



5. Configure Routing

- AWS automatically creates route tables for your subnets.
- For public subnets, ensure a route to the Internet Gateway (0.0.0.0/0 pointing to the IGW) is present
- For private subnets, using a NAT Gateway, ensure a route to the NAT Gateway (IPV4 CIDR of private subnets pointing to the NAT Gateway) is present

6. Configure Security

- Create Security Groups to control inbound and outbound traffic for instances within your subnets (Project-Security-Group)
- Create Security Groups to allow traffic for MySQL server (Project-Security-Group)

Step 2: EC2 Instance Setup

i) Navigate to EC2 in Console

ii) Create 4 instances separately & name as (public-instance-1, public-instance-2, private-instance-1 and private-instance-2)

iii) Select application and OSI (**Ubuntu**) for all instances

iv) Select t3.micro for all instances

v) Create one key pair and use one key pair for all instances

vi) Edit Network settings

a) Click on VPC select created VPC instead of default VPC

b) Select subnets for specific instances as same as subnets for (ex:- instance as public-instance-1 and public -instance-2)

c) Enable auto assign ip

vii) Create one security group and use this for all instances

a) SSH-22 port, anywhere – 0.0.0.0/0

b) HTTP-80 port, anywhere – 0.0.0.0/0

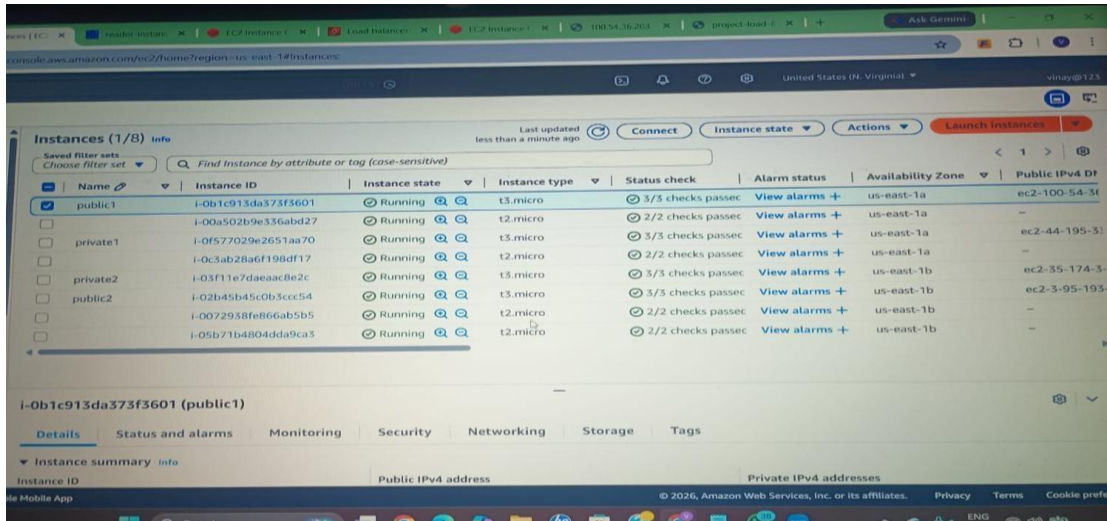


Fig 2: EC2 Instances

1. To create an image (Amazon Machine Image - AMI) of an EC2 instance

- Access the EC2 Dashboard
- Select the Instance: In the navigation pane, choose "Instances" and then select the specific EC2 instance from which you want to create an AMI->Actions->Images and templates
- Initiate Image Creation
- Configure AMI Details

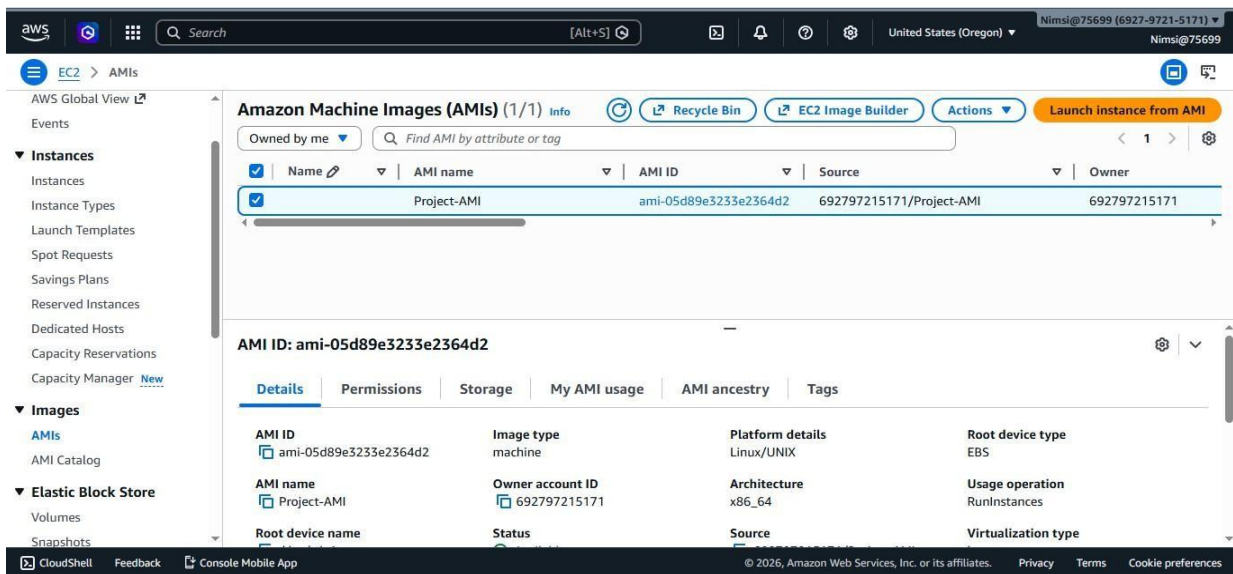


Fig 2.1: AMI (Image)

- **Launch Template**
 - Navigate to the EC2 Dashboard.
 - Select Launch Templates.
 - Choose Launch Template.
 - Create Template (Template)

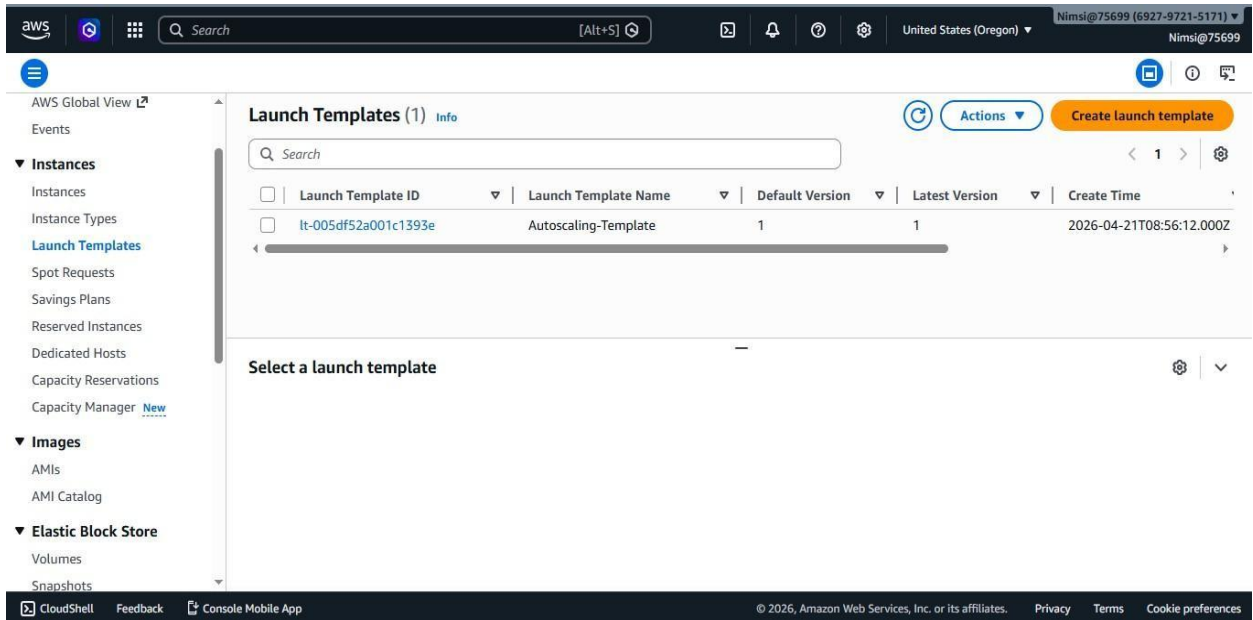


Fig 2.2: Template

- **Configure a Target Group**
 - Create Target Group: Choose Target Groups then "Create target group"
 - Basic Configuration:
 - Choose a target type: Select "Instances" to register EC2 instances or "IP addresses" to register targets by their IP
 - Protocol and Port: Define the protocol (e.g., HTTP, HTTPS) and port on which the targets will receive traffic.
 - Health checks: Configure health check settings to ensure the ALB only routes traffic to healthy targets.
 - Register targets: After creating the target group, navigate to its details and register the EC2 instances or IP addresses that will serve as targets for the ALB

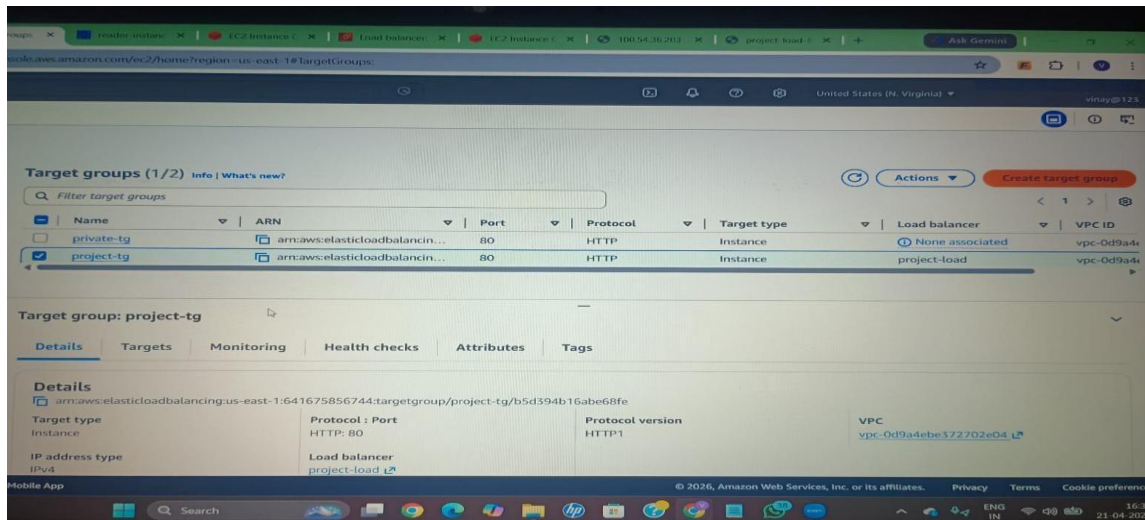


Fig 2.3: Target Groups

- **Configure the Application Load Balancer and a Listener:**
 - Create Load Balancer: In the EC2 console, under "Load Balancing," choose "Load Balancers," then "Create load balancer." Select "Application Load Balancer" and choose Create
 - Create 2 load balancers one for web tier, another for app tier

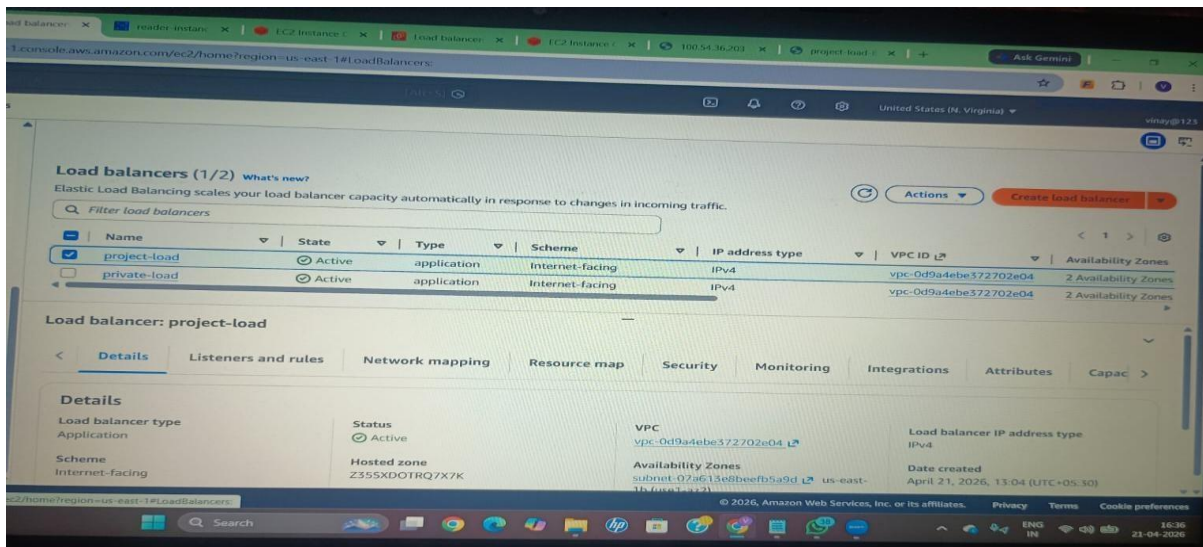


Fig 2.4: Load Balancers

- **Create an AWS Auto Scaling:**
 - Launch Template
 - Auto Scaling Group
 - Scaling Policies
 - Desired Capacity
 - Minimum/Maximum Size
 - Attach Load Balancers

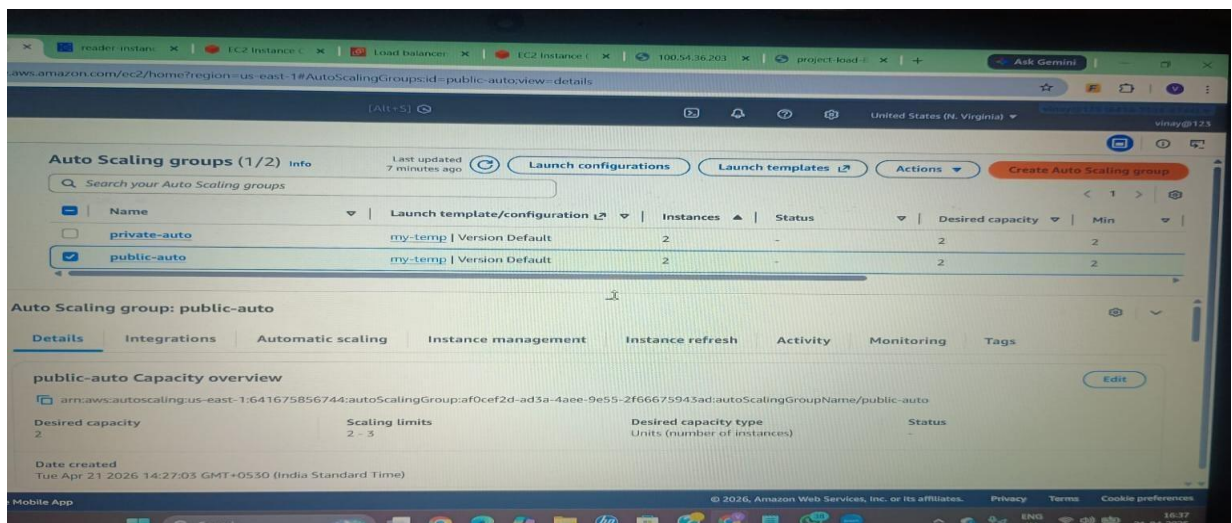


Fig 2.5: Auto Scaling

- After creating auto scaling we get extra ec2 instances which we gave the desired capacity of auto scaling
- Auto scale manages the servers for increasing and decreasing the server and also performance by adding extra CPU's when needed

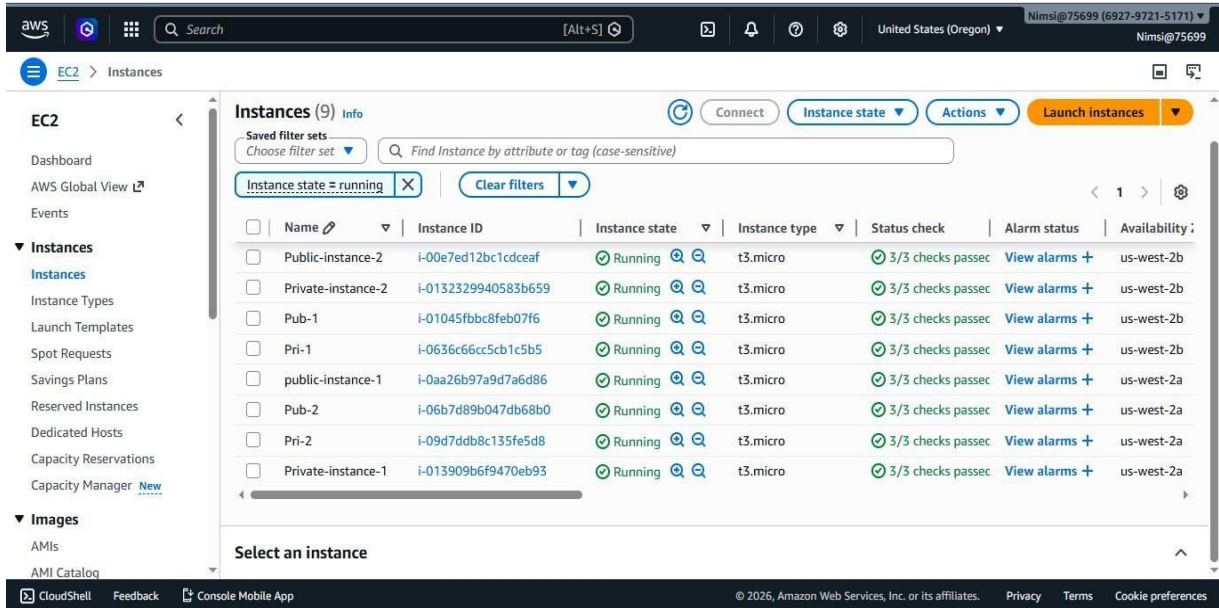


Fig 2.5.1: Auto Scaling

Step 3: Database Setup

1. Subnet Groups:

We need to create one subnet group for Database tier to create RDS database within the subnet group->project-subnet-group

Steps to create an RDS DB subnet group:

- Open the Amazon RDS console
- Initiate creation
- Provide details
- Select subnets
- Create the subnet group

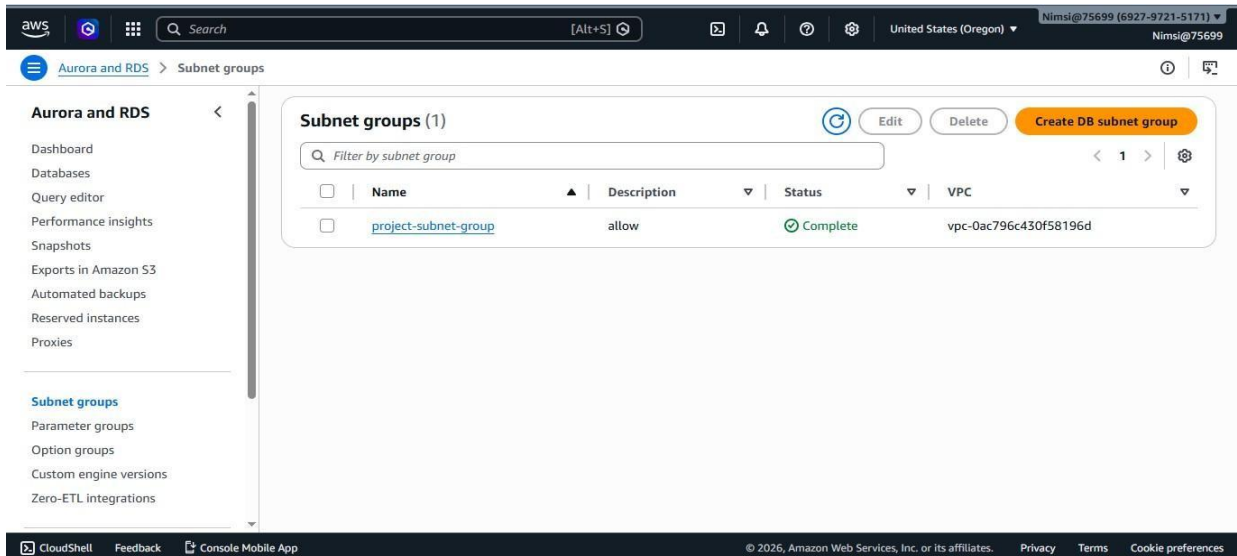


Fig 3.1: Subnets Group

2. Create RDS:

Creating an Amazon Relational Database Service (RDS) database involve several steps within the AWS Management Console

- Access the RDS Console
- Create Database
- Choose Creation Method and Engine (MySQL)
- Configure Database Settings (Multi-AZ (2 instances))
- Configure Connectivity (Project-Subnet-Group)
- Initial database name (project-database)
- Click on self-managed->password->confirm password->public access: yes
- Create RDS Proxy->click on all log exports

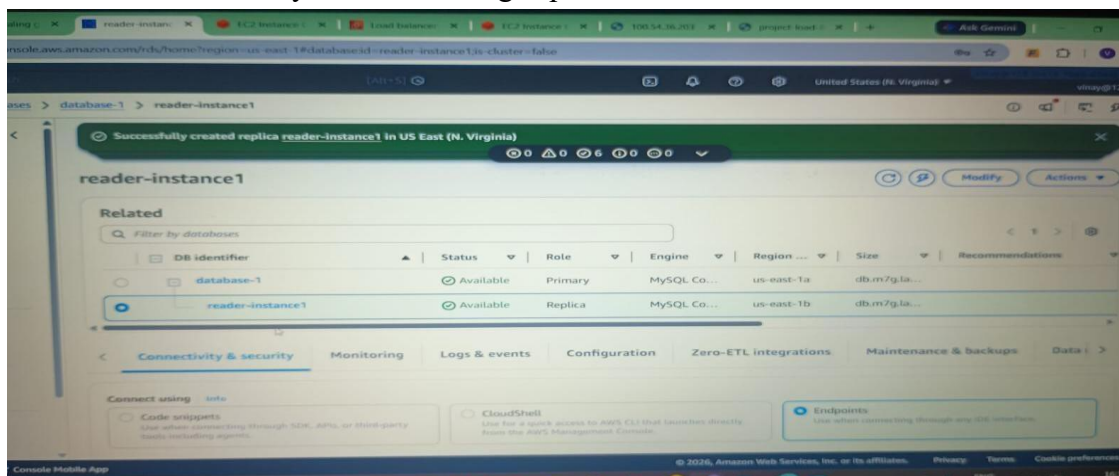


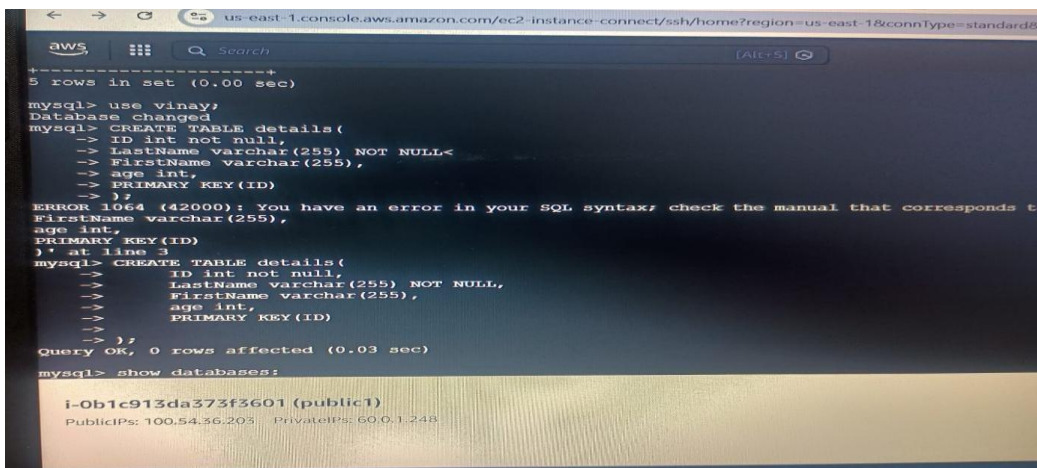
Fig 3.2: RDS Database

3. Connect and Manage:

- Once the database status changes to "Available," you can connect to it using a database client (e.g., MySQL Workbench) and the master user credentials.

Connect PUB-EC2-1 to PVT-EC2-1

- First connect web instance through Gitbash, build a connection and make sure root user (**sudo -i**).
- Save the key pair inside the web instance with the same name used in the private instance after connecting through ssh.
- Change the permissions of user to read only (chmod 400 project key.pem)
- Copy the ssh client link of PVT-EC2-1 to replace the public IP with



```
aws us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=us-east-1&connType=standard&...
mysql> use vinay;
Database changed
mysql> CREATE TABLE details(
  ID int not null,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  age int,
  PRIMARY KEY(ID)
);
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to
  FirstName varchar(255),
  age int,
  PRIMARY KEY(ID)
); at line 3
mysql> CREATE TABLE details(
  ID int not null,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  age int,
  PRIMARY KEY(ID)
);
Query OK, 0 rows affected (0.03 sec)
mysql> show databases;
i-0b1c913da373f3601 (public1)
PublicIPs: 100.54.36.203 PrivateIPs: 60.0.1.248
```

private IP (app instance) and then make connection successfully.

Installation & Connection to Database

- For connecting to RDS we need to install MySQL server in App instance.
 - Follow the following commands to install MySQL in App server
- ➔ Sudo apt update
 - ➔ Sudo apt install mysql-server
 - ➔ Sudo systemctl start mysql.server
 - ➔ mysql -version
- After installing the MySQL server in APP tier we need to make connection to MySQL with the **RDS Endpoint** as shown in below
 - ➔ mysql -h project-database.c7s4egw441bf.us-west-2.rds.amazonaws.com -u admin -p
 - ➔ After the above command enter the self-managed password which was already created during the creation of Database

- Here we need to create one Database and connect it to create a table

```

mysql> use vinay
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_vinay |
+-----+
| details          |
+-----+
1 row in set (0.01 sec)

mysql> select *FROM details;
+----+-----+-----+-----+
| ID | LastName | FirstName | age |
+----+-----+-----+-----+
| 80 | vinay1   | repalle   | 23  |
+----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>

```

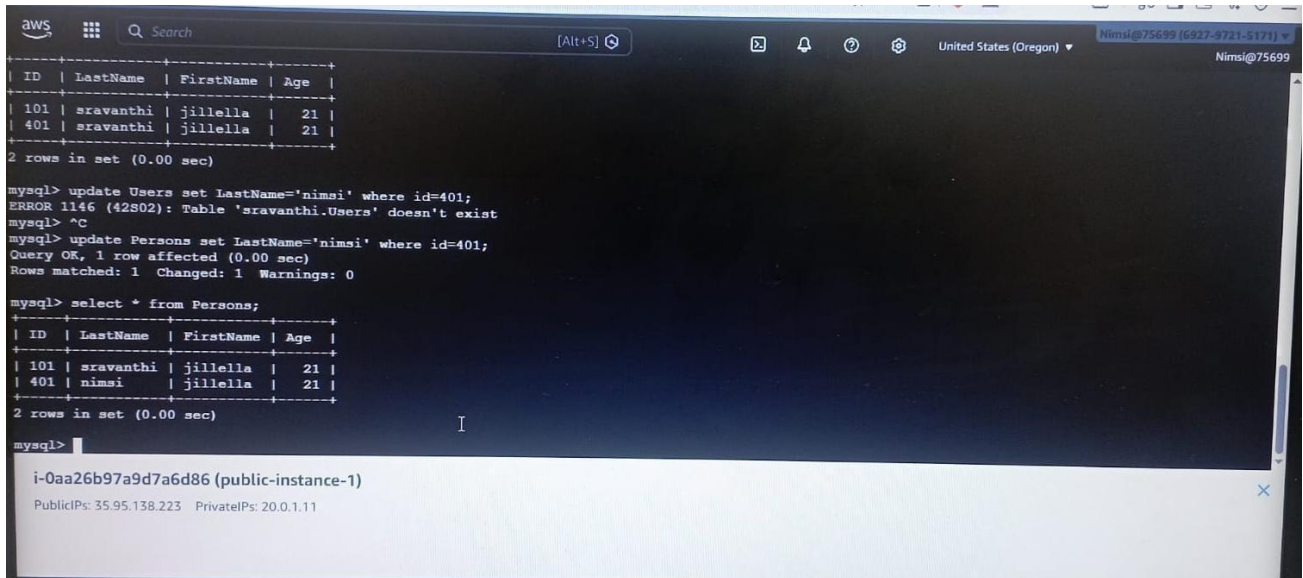
i-02b45b45c0b3ccc54 (public2)
PublicIPs: 3.95.193.89 - PrivateIPs: 60.0.2.178

CloudShell Feedback Console Mobile App

T-storms At night

- Create database sravanthi;
- Use sravanthi;
- CREATE TABLE Persons (id INT AUTO_INCREMENT PRIMARY KEY, LastName VARCHAR(100), FirstName VARCHAR(100));
- INSERT INTO Persons (LastName, FirstName) VALUES ('sravanthi', 'jillella');
- Update Persons set name='sravanthi' where id=101;
- Select * from Persons;

Successfully we accessed Persons instance



```
aws
Search [Alt+S]
United States (Oregon)
Nimsi@75699 (6927-9721-5171)
Nimsi@75699

+----+-----+-----+-----+
| ID | LastName | FirstName | Age |
+----+-----+-----+-----+
| 101 | sravanthi | jillella | 21 |
| 401 | sravanthi | jillella | 21 |
+----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> update Users set LastName='nimsi' where id=401;
ERROR 1146 (42S02): Table 'sravanthi.Users' doesn't exist
mysql> ^C
mysql> update Persons set LastName='nimsi' where id=401;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from Persons;
+----+-----+-----+-----+
| ID | LastName | FirstName | Age |
+----+-----+-----+-----+
| 101 | sravanthi | jillella | 21 |
| 401 | nimsi | jillella | 21 |
+----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>

i-0aa26b97a9d7a6d86 (public-instance-1)
PublicIPs: 35.95.138.223  PrivateIPs: 20.0.1.11
```

Summary

The three-tier architecture is a software design approach that divides an application into three layers: presentation layer, application (business logic) layer, and data layer.

Using services like Load Balancing, EC2, Auto Scaling, RDS, and VPC, it delivers high scalability, availability, security, and fault tolerance. Each tier is placed in its own subnet with proper security groups, which improves performance and protects resources.

Three-tier architecture is widely used in modern applications because it provides a clear structure, better control, and efficient system management, making it ideal for large and complex projects.